This example can be executed with the command:
`VITE_NAME=VITE_NAME=SearchComponentExample npm run dev.`

To ensure React Hooks work correctly and avoid runtime errors, React enforces two fundamental rules when using Hooks:

- Hooks must be called at the top level of a React function component or a custom Hook. They cannot be called inside loops, conditions, or nested functions. This rule ensures that Hooks are always executed in the same order during every render, allowing React to correctly preserve state across renders.
- Hooks can only be invoked from React function components or custom Hooks. They cannot be called from regular JavaScript functions or class components. This ensures that all stateful logic is tied to React's rendering process.

NOTE

React relies on the order of Hook calls to associate state and behavior with specific components. To avoid bugs and ensure correct functionality:

- Do not call Hooks inside conditions or loops.
- Do not call Hooks after a conditional return statement.
- Do not call Hooks in event handlers.
- Do not call Hooks in class components.
- Do not call Hooks inside functions passed to useMemo, useReducer, or useEffect.
- Do not call Hooks inside try/catch/finally blocks.

Initial question (advanced): "How would you implement a custom React Hook called `useDebounce` to optimize a search bar, and what are the design considerations for ensuring it works efficiently?"

Analysis: This question assesses a candidate’s ability to apply React Hooks to build a custom hook like `useDebounce` and to discuss design considerations such as type safety, memory management, and generic reusability. It also evaluates the candidate’s communication skills in clarifying requirements and